
Documentation technique

Système d'acquisition d'images d'insectes

Mame Diarra Wade

Stage BUT3

Laboratoire Chrono-Environnement — UMR 6249

Tuteur professionnel : Pierre - Emmanuel
Leni

Tutrice académique : Christine
Bolou-Chiaravalli

Période : 08/04/2026 – 03/06/2026

Juin 2026

Table des matières

1	Contexte et objectif	3
2	Architecture générale	3
3	Environnement technique	3
4	Bibliothèques et outils	3
4.1	OpenCV (cv2)	3
4.2	Pillow (PIL)	4
4.3	subprocess	4
4.4	threading	4
4.5	gphoto2 (Linux)	4
4.6	digiCamControl (Windows)	4
4.7	v4l2-ctl	4
5	Algorithmes et logique de traitement	5
5.1	Nommage automatique des fichiers	5
5.2	Création automatique des dossiers	5
5.3	Focus stacking	6
5.4	Prévisualisation temps réel	7
6	Problèmes rencontrés et solutions	7
7	Perspectives	8

1 Contexte et objectif

Ce projet s'inscrit dans un stage de troisième année de BUT Informatique au laboratoire Chrono-Environnement de Montbéliard (avril–juin 2026). L'objectif est de constituer un dataset d'images d'insectes de haute qualité pour entraîner un modèle d'intelligence artificielle de reconnaissance d'espèces.

Trois appareils sont pilotés depuis une seule interface graphique : un Canon EOS R7, une caméra de microscopie Jeulin e-Mago, et un smartphone Android. Le logiciel gère le nommage automatique des fichiers et intègre un algorithme de focus stacking.

2 Architecture générale

Fichier	Rôle
<code>interface.py</code>	Point d'entrée. Interface graphique Tkinter bilingue FR/EN.
<code>text.py</code>	Textes FR/EN et guide d'utilisation séparés du code.
<code>acquisition/canon.py</code>	Contrôle Canon via gphoto2 (Linux) ou digiCamControl (Windows).
<code>acquisition/jeulin.py</code>	Capture Jeulin via OpenCV VideoCapture (protocole UVC).
<code>acquisition/android.py</code>	Contrôle Android via ADB (Android Debug Bridge).
<code>acquisition/nommage.py</code>	Nommage structuré et création automatique des dossiers.
<code>traitement/stacking.py</code>	Focus stacking : enfuse en priorité, fallback Mertens OpenCV.

3 Environnement technique

Système	Ubuntu 20.04 LTS / Linux Mint 22.3
Langage	Python 3.8 dans un environnement virtuel (venv)
Interface graphique	Tkinter (inclus dans Python)
Versioning	Git + GitHub

4 Bibliothèques et outils

4.1 OpenCV (cv2)

Traitement d'image et capture vidéo. Utilisé pour : capturer les images de la Jeulin via VideoCapture, l'algorithme de focus stacking (fallback Mertens), et la prévisualisation live.

La Jeulin utilise le protocole UVC (USB Video Class), standard universel des webcams, reconnu automatiquement sous Linux comme `/dev/videoX`.

Listing 1 – Capture d'image avec OpenCV

```
1 import cv2
2 cap = cv2.VideoCapture(index_jeulin)
3 cap.set(cv2.CAP_PROP_BUFFERSIZE, 1) # éviter les frames en retard
4 ok, frame = cap.read()
5 frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) # BGR -> RGB
6 cv2.imwrite(chemin_fichier, frame)
7 cap.release()
```

4.2 Pillow (PIL)

Affichage des images dans Tkinter. OpenCV produit du BGR, Pillow attend du RGB. `ImageTk.PhotoImage` convertit en format affichable par Tkinter.

4.3 subprocess

Lance des commandes système depuis Python : `gphoto2`, `v4l2-ctl`, `ADB`.

4.4 threading

Lance les captures en arrière-plan pour que l'interface reste réactive. `daemon=True` garantit l'arrêt des threads à la fermeture. Tkinter n'est pas thread-safe : toute modification de widget depuis un thread secondaire passe par `fenetre.after()`.

4.5 gphoto2 (Linux)

Outil CLI contrôlant le Canon via USB. Quand téléphone et Canon sont branchés simultanément, `gphoto2` sans `--port` prend le premier appareil détecté, souvent le téléphone. On utilise `--port usb:XXX,YYY` pour cibler explicitement le Canon.

4.6 digiCamControl (Windows)

Remplace `gphoto2` sur Windows. CLI avec `/capture /filename chemin.jpg`.

4.7 v4l2-ctl

Contrôle natif des propriétés V4L2 de la caméra Jeulin. OpenCV traduit incorrectement les plages de valeurs du driver. `v4l2-ctl` envoie la valeur native sans conversion.

Listing 2 – Réglage gamma via v4l2-ctl

```
1 v4l2-ctl -d /dev/video2 --set-ctrl=gamma=200
```

```
2 v4l2-ctl -d /dev/video2 --set-ctrl=brightness=0
```

5 Algorithmes et logique de traitement

5.1 Nommage automatique des fichiers

Algorithme de nommage

Le nommage encode toutes les métadonnées directement dans le nom de fichier. Le modèle IA peut lire l'espèce, l'angle, l'appareil sans base de données externe.

Convention :

espece_indXX_camAppareil_angleAngle_photoXX.extension

Exemple :

scolyte1_ind01_camCanon_angleDorsal_photo03.jpg

Algorithme d'incrémentation :

Boucle `while` qui génère le nom avec le numéro courant, teste si le fichier existe avec `os.path.exists()`, incrémente si oui. Garantit qu'aucune photo n'est jamais écrasée. Complexité $O(n)$ où n est le nombre de photos existantes.

Listing 3 – Algorithme d'incrémentation du numéro de photo

```
1 def prochain_numero_photo(dossier, espece, num_individu,
2                             appareil, angle, extension):
3     numero = 1
4     while True:
5         nom = generer_nom_fichier(espece, num_individu,
6                                     appareil, angle, numero,
7                                     extension)
8         if not os.path.exists(os.path.join(dossier, nom)):
9             return numero
10        numero += 1
```

5.2 Création automatique des dossiers

Listing 4 – Création automatique de la structure de dossiers

```
1 def generer_chemin_dossier(dossier_base, appareil, espece,
2                             num_individu):
3     chemin = os.path.join(dossier_base, appareil, espece,
4                             f"individu_{num_individu:02d}")
5     # exist_ok=True : ne plante pas si le dossier existe deja
6     os.makedirs(chemin, exist_ok=True)
7     return chemin
```

5.3 Focus stacking

Algorithme de focus stacking

Problème : À fort grossissement, la profondeur de champ est très faible. Une seule photo ne peut pas avoir tout l'insecte net.

Solution : Combiner plusieurs photos prises à des mises au point différentes pour produire une image entièrement nette.

Méthode principale — enfuse :

Outil open source (paquet Ubuntu). Fusion multi-bandes par pondération du contraste (`--contrast-weight=1`). Algorithme équivalent à Helicon Focus (logiciel professionnel payant).

Fallback — AlignMTB + Mertens (OpenCV) :

Utilisé si enfuse n'est pas installé.

- **AlignMTB** : aligne les images par seuillage médian de luminosité. Corrige les décalages x/y. Rapide car travaille sur des images binarisées.
- **Mertens** : calcule pour chaque pixel un score de netteté basé sur le contraste local (Laplacien), la saturation et l'exposition. Combine via pyramides laplaciennes pour éviter les artefacts.

Listing 5 – Focus stacking — méthode Mertens (fallback)

```
1 def _stacking_mertens(liste_chemin, chemin_sortie):
2     images = [cv2.imread(c) for c in liste_chemin]
3     # Alignement : corrige les micro-décalages dus au focus manuel
4     aligneur = cv2.createAlignMTB()
5     aligneur.process(images, images)
6     # Fusion Mertens : pondération par netteté (contraste local)
7     fusionneur = cv2.createMergeMertens()
8     resultat_float = fusionneur.process(images)
9     # Conversion float32 (0-1) vers uint8 (0-255)
10    resultat = np.clip(resultat_float * 255, 0, 255).astype("uint8")
11    cv2.imwrite(chemin_sortie, resultat)
```

5.4 Prévisualisation temps réel

Algorithme de prévisualisation

La prévisualisation Jeulin utilise `fenetre.after(16, _tick_preview)` — appel toutes les 16ms dans le thread principal Tkinter (60 fps théoriques).

Pourquoi `after()` et pas un thread secondaire ?

Tkinter n'est pas thread-safe. Modifier un widget depuis un thread secondaire cause des crashes. `after()` planifie l'exécution dans le thread principal, évitant la synchronisation inter-thread.

CAP_PROP_BUFFERSIZE=1 : élimine les frames en retard dans le buffer V4L2. On lit toujours l'image la plus récente.

Image.NEAREST : 3× plus rapide que LANCZOS pour le redimensionnement, imperceptible à cette taille.

6 Problèmes rencontrés et solutions

Problème	Solution
EDSDK : DirItem-Created jamais déclenché sur Linux	EDSDK abandonné. Migration vers gphoto2 CLI (Linux) et digiCamControl (Windows).
gphoto2 prend le téléphone à la place du Canon	<code>_detecter_port_canon()</code> parse la sortie de <code>--auto-detect</code> et cible <code>--port usb:XXX,YYY</code> .
Conflit USB Canon + téléphone (gvfs)	<code>kill gvfs-gphoto2, gvfsd-mtp</code> + règle udev permanente (<code>ENV{ID_MEDIA_PLAYER}=1</code>).
Erreur 0x2019 PTP Device Busy	Objectif en AF. Message affiché dans l'interface. Réglage physique sur l'objectif (MF).
ADB version mismatch (1.0.39 vs 1.0.41)	<code>sudo apt install adb</code> sur Linux Mint 22.3 installe directement la version correcte.
Curseurs luminosité sans effet (plage OpenCV incorrecte)	Migration vers <code>v4l2-ctl</code> qui envoie la valeur native sans conversion.
Prévisualisation avec délai visible	<code>CAP_PROP_BUFFERSIZE=1</code> + lecture de 5 frames sans affichage au démarrage.
Jeulin « non branché » lors d'actualisations	Compteur <code>_jeulin_echecs</code> : statut changé seulement après 3 échecs consécutifs.
Preview ne redémarre pas après reconnexion	Mémorisation du chemin <code>/dev/videoX</code> . Arrêt/redémarrage propre si le chemin change.

7 Perspectives

- **Recadrage automatique** par zone anatomique (tête, thorax, élytres) selon l'angle de prise de vue, avec un modèle YOLO entraîné sur des insectes.
- **Base de données SQLite** pour la gestion du dataset (requêtes par espèce, équilibrage, statistiques).
- **Mode batch** : photographier automatiquement un individu à tous les angles.